FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

SMART CITY WATER ALLOCATION SYSTEM

Water Jug Search, Constraint Satisfaction and Rule-Based Diagnosis

Submitted by

ISRAEL P

Register Number: 113025148035

B.E. Computer Science and Engineering (AI & ML)

Second Year - Third Semester

Course Code: 25ML35T

Submitted to: Dr. Manoj Kumar D S, Associate Professor

Department: CSE (AI & ML)

Academic Year: 2026-2027

Project report generated from the current implementation and recorded test results.

Abstract

This project combines a classical Water Jug Problem solver with a prototype Smart City Water Allocation System. Breadth-First Search (BFS) and Depth-First Search (DFS) find sequences of jug operations. A constraint-satisfaction model allocates reservoir water to a hospital and two residential zones while respecting reservoir availability, pipeline limits and minimum daily supply constraints. A rule-based diagnosis module flags possible leaks or shortages from supplied indicators. The current automated test suite reports 16 passing tests. This is a software simulation, not a live water-control system.

1. Objectives

• Model the Water Jug Problem as a finite state-space search problem.

• Implement and test BFS and DFS solution paths.

• Allocate reservoir water to zones using constraint satisfaction and backtracking.

• Compare search effort with and without constraint propagation.

• Demonstrate transparent rule-based leak and shortage warnings.

• Document minimum-supply constraints, limitations and responsible-use considerations.

2. Introduction and Problem Formulation

The Water Jug Problem represents states as pairs (A, B), where A and B are the current amounts of water in two jugs. Legal actions include filling, emptying and pouring between jugs. A goal state has the target amount in at least one jug. BFS explores states level by level and returns a shortest path in an unweighted state graph; DFS explores a branch before backtracking and does not guarantee a shortest path.

The extension models water allocation as a constraint-satisfaction problem (CSP). Reservoir supplies are finite, pipeline flows have limits, and each zone has a minimum daily demand. The solver searches for an allocation satisfying the encoded constraints.

3. AI Concepts Used

State-space search: BFS and DFS explore reachable jug states.

Constraint satisfaction: reservoir-to-zone flows must satisfy supply, pipeline and minimum-demand constraints.

Backtracking: the allocation solver explores candidate assignments and reverses choices when constraints cannot be satisfied.

Constraint propagation: impossible candidate assignments are pruned earlier, reducing search effort in the recorded scenarios.

Rule-based reasoning: input indicators activate diagnosis rules and produce a label, certainty factor and explanation.

4. System Architecture

Component	Responsibility
project_config.py	Stores the register number and configured variant ID.
solver.py / variant.py	Defines the jug problem and BFS/DFS search.

Component	Responsibility
water_allocation.py	Defines zones, reservoir scenarios, pipeline limits and CSP search.
diagnosis.py	Applies rule-based leak and shortage diagnosis.
city_integration.py	Presents jug search and city allocations in one demonstration.
tests/	Automated checks for search, allocation, diagnosis and integration.
results/	Stores recorded demonstration outputs and benchmark data.

5. Methodology

The jug solver represents each state as a pair of integer volumes. BFS uses a queue and DFS uses a stack; visited-state handling prevents unbounded revisiting. The configured instance uses jug capacities of 4 L and 5 L with a 1 L target. This configuration is controlled by the local variant settings and should be confirmed against the instructor's assignment.

The allocation model uses 10 kL units. Its minimum-supply rule is 10 litres per person per day. The encoded zone populations produce minimums of 3 units for the Hospital, 3 for Residential East and 2 for Residential West. Reservoir and pipeline constraints are applied during search.

Scenario	North supply	South supply	Hospital minimum	East minimum	West minimum
Normal	9 units	6 units	3 units	3 units	2 units
Drought	5 units	4 units	3 units	3 units	2 units

6. Rule-Based Diagnosis

Condition	Output	Certainty factor
Low pressure AND high flow	LeakSuspected	0.8
Low reservoir AND insufficient supply	WaterShortageSuspected	0.9
No listed rule matches	NoKnownIssueDetected	0.7

Example trace: low pressure = true and high flow = true activates the LeakSuspected rule with CF 0.8. These certainty factors are illustrative rule weights, not calibrated probabilities and not proof of a real leak. Field use would require validated sensors, calibration and human review.

7. Implementation and Reproducibility

The project is implemented in Python. Tests are run with pytest. From the repository root, use the commands below in an environment where the dependencies in requirements.txt are installed.

```
python -m pytest -v
python -m src.smart_city_demo
python -m src.city_integration
python src/main.py --roll-number 113025148035 --variant-id 1 --method bfs
python src/main.py --roll-number 113025148035 --variant-id 1 --method dfs
```

The integration output is recorded in results/city_integration.txt; the standalone allocation and diagnosis demonstration is recorded in results/smart_city_demo.txt. Confirm that these outputs match the current code before submission.

8. Results and Observations

Scenario	Hospital	Residential East	Residential West	North used	South used
Normal	60 kL/day	50 kL/day	40 kL/day	9/9 units	6/6 units
Drought	40 kL/day	30 kL/day	20 kL/day	5/5 units	4/4 units

Scenario	Propagation	Nodes visited	Pruned branches
Normal	Disabled	475	469
Normal	Enabled	6	0
Drought	Disabled	1377	1371
Drought	Enabled	7	1

The recorded node counts show substantially less search effort with propagation for these two scenarios. Node counts are not execution times and should not be generalized beyond the tested model.

Jug search method	Recorded operations	Expanded nodes	Observation
BFS	2	6	Shortest path for the configured unweighted problem.
DFS	12	14	Valid path, but longer than BFS in this instance.

The automated suite completed with 16 passed tests in 0.06 seconds in the recorded run. This includes tests for jug search, configured integration, diagnosis rules and allocation constraints.

9. Responsible AI, Limitations and Future Work

The encoded minimum daily supply provides a baseline service constraint. The model reports zone allocations and reservoir usage so that results can be inspected. Hospital priority currently affects search ordering; it is not a formal optimization objective and does not guarantee a globally fair allocation beyond the encoded constraints.

The scenarios and diagnosis inputs are simulated. The certainty factors are illustrative. The model does not account for real-time demand, water quality, losses, changing population, energy costs, uncertainty or physical network hydraulics. The jug sequence is a metering illustration and must not be interpreted as a physical valve command.

Future work could add validated real-world data, explicit fairness objectives, uncertainty-aware diagnosis, a user interface, more scenarios, and a human approval step before any operational decision.

10. Conclusion

The project demonstrates how classical search can be integrated with constraint satisfaction and rule-based reasoning in a simulated Smart City water allocation use case. The recorded tests verify the current implemented behaviors, while the scenario results illustrate the effect of constraint propagation. Real deployment would require additional engineering, validated data and operational safeguards.

11. References

• Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th edition, Pearson.

• Python documentation: https://docs.python.org/3/

• pytest documentation: https://docs.pytest.org/

• ReportLab documentation: https://docs.reportlab.com/

Final Submission Checklist

■ Confirm the assigned variant ID with the instructor.

■ Run the tests again and retain the final test output.

■ Check the generated report against the actual implementation and college template.

■ Add genuine screenshots and the actual GitHub repository URL if required by the template.

■ Verify Git history, meaningful commits on at least two calendar days, and the v1.0 tag.

■ Ensure .venv, caches and backup files are excluded from the submitted repository.